Documentazione

Materia: TPSIT (Tecnologie e Progettazione di Sistemi Informatici e di Telecomunicazioni).
Membri gruppo: Spartano Alessio, Warnakulasuriya Ashan, Lorenzo Santangelo, Zanette Andrea.

INDICE

1. INTRODUZIONE……..……..…

1.1 SETUP……..……...……..……..……... PAG 3

1.2. OBIETTIVO……..……..……...… PAG 3

1.3. CARATTERISTICHE……..……..……..……... PAG 4

1.4. STRUTTURA DIRECTORY……... PAG 5

1.5 CONFIGURAZIONE……..……..……...……... PAG 5

2. WEB API……..……..……...

2.1 INFORMAZIONI GENERALI……. PAG 6

2.2 LISTA RISORSE……..……. PAG 6

2.3 RELAZIONI TRA RISORSE……... PAG 6

3. AUTENTICAZIONE……..……..……..……

3.1 LIVELLI DI AUTORIZZAZIONE……. …… PAG 7

4. SCRIPTS PHP……...……..

4.1. FLUSSO DEL FRAMEWORK…...…… PAG 8

4.2. ENTRY POINT…..……..... PAG 8

4.3 DESCRIZIONE GENERALE ROUTER……...…. PAG 8

4.4. METODI PER LA GESTIONE DI RICHIESTE……...…… PAG 9

5. DATABASE……..……..…….

5.1. SCHEMA LOGICO……..…….. PAG 10

5.2 SCHEMA ER SEMPLICE……..…… PAG 10

5.2 SCHEMA ER COMPLETO…… PAG 11

1. INTRODUZIONE

1.0 Setup

- Per visualizzare la documentazione in versione sito (metodo consigliato) estrarre file “documentazione.zip” e hostarne il sito all’iinterno. In caso si può anche aprire direttamente “index.html” ma I reindirizzamenti richiederanno ogni volta di selezionare “l’index.html” della pagina.
- Il webservice ha come requisiti PHP=8.12.2, COMPOSER e MARIADB.
- Il progetto richiede l’utilizzo di composer per l’installazione delle librerie (JWT, Swagger, Var exporterer):
  - composer install
  - composer dump-autoload

1.1 Obiettivo

Questo progetto di gruppo ha lo scopo di integrare e applicare le conoscenze e le competenze acquisite nelle discipline di Informatica (per database) e TPSIT.

Si lavora in gruppo per progettare, sviluppare e documentare un'applicazione software che risponda a specifiche esigenze tra le quali quella di realizzare un web-service in php con framework assestante progettato “from scratch”.

L’obiettivo che il team si è dato per questo progetto è quello di sviluppare un servizio per la progettazione di computer fissi personalizzati. Il servizio permette di selezionare progressivamente diversi componenti (quali il processore, scheda madre, RAM, alimentatore e storage), visualizzare le componenti compatibili con le scelte precedenti e salvare le configurazioni nell’account personale dello user.

1.2 Caratteristiche

Il progetto ha molte caratteristiche ma, tra le più importanti, ci sono:

- Consultare un catalogo di componenti hardware (CPU, GPU, RAM, ecc.) organizzati per categoria e produttore.
- Creare e gestire build (configurazioni PC) personalizzate, con verifica automatica della compatibilità.
- Gestire categorie di componenti con le relative specifiche tecniche (es. socket, frequenza).
- Gestire i produttori dei componenti.
- Autenticarsi tramite JWT (cookie `token`) con due livelli di accesso: utente normale e owner (amministratore).

Caratteristica	Descrizione
Framework	Programmato da zero, secondo standard moderni, interamente autonomo e riutilizzabile.
Caching	Alta scalabilità grazie alle prestazioni avanzate garantite dal sistema di caching integrato.
Endpoints	Flessibilità e semplicità per programmare endpoint custom in base alle proprie esigenze.
OpenAPI	Ottima compatibilità grazie alle API conformi alle specifiche OpenAPI.
Swagger UI	Integrazione di Swagger UI che presenta le API in modo interattivo ed efficace.
Authentication	Sistema di gestione delle autorizzazioni sulle richieste tramite JWT.

1.3 Struttura Directory

```
htdocs
├── index.php                  ← entry point unico (eseguito dal web server)
├── .htaccess                  ← rewrite engine: tutte le richieste → index
├── config/
│   └── config.yaml            ← configurazione (debug, hosts, path)
├── app/
│   ├── authorization/         ← gestione JWT e auth
│   │   └── Authorization.php
│   ├── middlewares/           ← middleware di autenticazione
│   │   ├── AuthMiddleware.php
│   │   └── OwnerAuthMiddleware.php
│   └── routes/                ← controller per ogni risorsa
│       ├── builds/
│       ├── categories/
│       ├── components/
│       ├── manufacturers/
│       └── profile/
├── framework/                 ← codice generale (Router, Request, Response…)
└── db.sql                     ← file SQL per la creazione del database
```

1.4 Configurazione

Il file config/config.yaml contiene i dati di configurazione del web service:

```yaml
app:
  debug: false
  allowed_hosts:
    - localhost
    - 127.0.0.1
  docs:
    /docs/swagger.html


directories:
  controllers: /app/routes
  middlewares: /app/middlewares
  static: /static
```

2. WEB API

2.1 Informazioni generali

- URL del web service: http://localhost
- Documentazione interattiva dell'intera API (attraverso swagger UI) disponibile all'avvio su: http://localhost/docs. Essa contiene Informazioni su tutte le risorse, le loro operazioni CRUD consentite e tutti gli altri dettagli inerenti.

2.2 Lista risorse

Risorsa	URI base	Descrizione
Users	/api/profile, /api/login, /api/register	Utenti registrati; gestione autenticazione
Manufacturers	/api/manufacturers	Produttori hardware (AMD, Intel, ASUS, …)
Categories	/api/categories	Categorie di componenti (CPU, GPU, RAM, …) con relative specifiche
Components	/api/components	Componenti hardware con specifiche tecniche e prezzo
Builds	/api/builds	Configurazioni PC create dagli utenti
Build Components	/api/builds/{buildId}/ components	Componenti associati a una build, con verifica compatibilità
Rules	/api/rules	Regole di compatibilità tra componenti

2.3 Relazioni tra risorse

Entità	Relazione	Chiave
Build_Components	contiene →	component_id
Components	← contenuto in	component_id

3. AUTENTICAZIONE

Il web service implementa due livelli di accesso, verificati tramite cookie JWT (token).

3.1. Livelli di Autorizzazione

Middleware	Requisito	Risposta in caso di errore
(nessuno)	Accesso libero	—
AuthMiddleware	Utente autenticato (login effettuato)	401 Unauthorized
OwnerAuthMiddleware	Utente autenticato e con flag is_owner = true	401 se non loggato, 403 Forbidden se loggato ma non owner

4. SCRIPTS PHP

Il framework usa il paradigma Object-oriented. Si fa riferimento a file PHP che fungono da classi distinte.

4.1. Flusso del framework

1. Creazione oggetto Request
2. Gestione del CORS
3. Determinazione rotta corretta tramite cache
4. Istanziamento del controller appropriato
5. Esecuzione dell'azione richiesta
6. Restituzione della risposta al client

4.2 Entry point

Script	Quando viene richiamato
index.php	Viene richiamato per ogni richiesta HTTP al web service. Carica la configurazione da config/config.yaml, inizializza il Router e FileHandler, e delega la gestione della richiesta al Router.

4.3 Descrizione generale Router

Il Router (framework/Router.php) riceve la richiesta da index.php, individua il controller corretto in base a metodo HTTP e percorso URI, applica i middleware di autorizzazione e delega l'esecuzione al controller appropriato.

4.4 Metodi per la gestione di richieste

Tutte le classi dei metodi citati si trovano in /framework.

Metodo	Funzione
Router::init()	Salva la configurazione delle directory da Config. Chiama RouteBuilder::build in caso di modifiche alle route.
RouteBuilder::build()	Scansiona ricorsivamente una directory, costruisce un albero di route indicizzato e lo salva in cache come array associativo in framework/cache/routes.php.
Router::handle()	Gestisce una richiesta HTTP cercando corrispondenza tra URI e route registrate. Istanzia Request, gestisce CORS, effettua match route, chiama runMiddlewares(), invoca Controller->manageRequest e invia la risposta.
runMiddlewares()	Chiama una catena di Middleware per validare la richiesta.
Controller->manageRequest()	Istanzia il controller (se necessario) e invoca il suo handler __invoke.

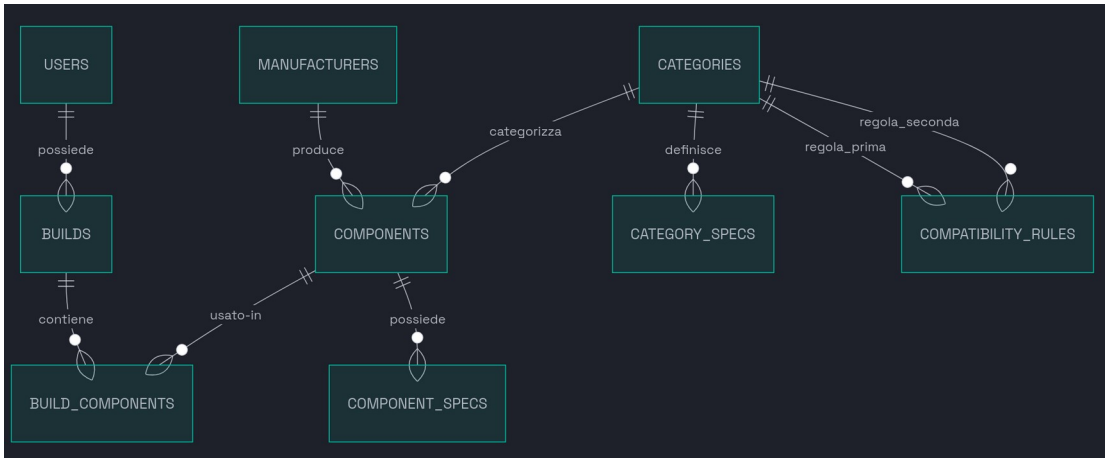
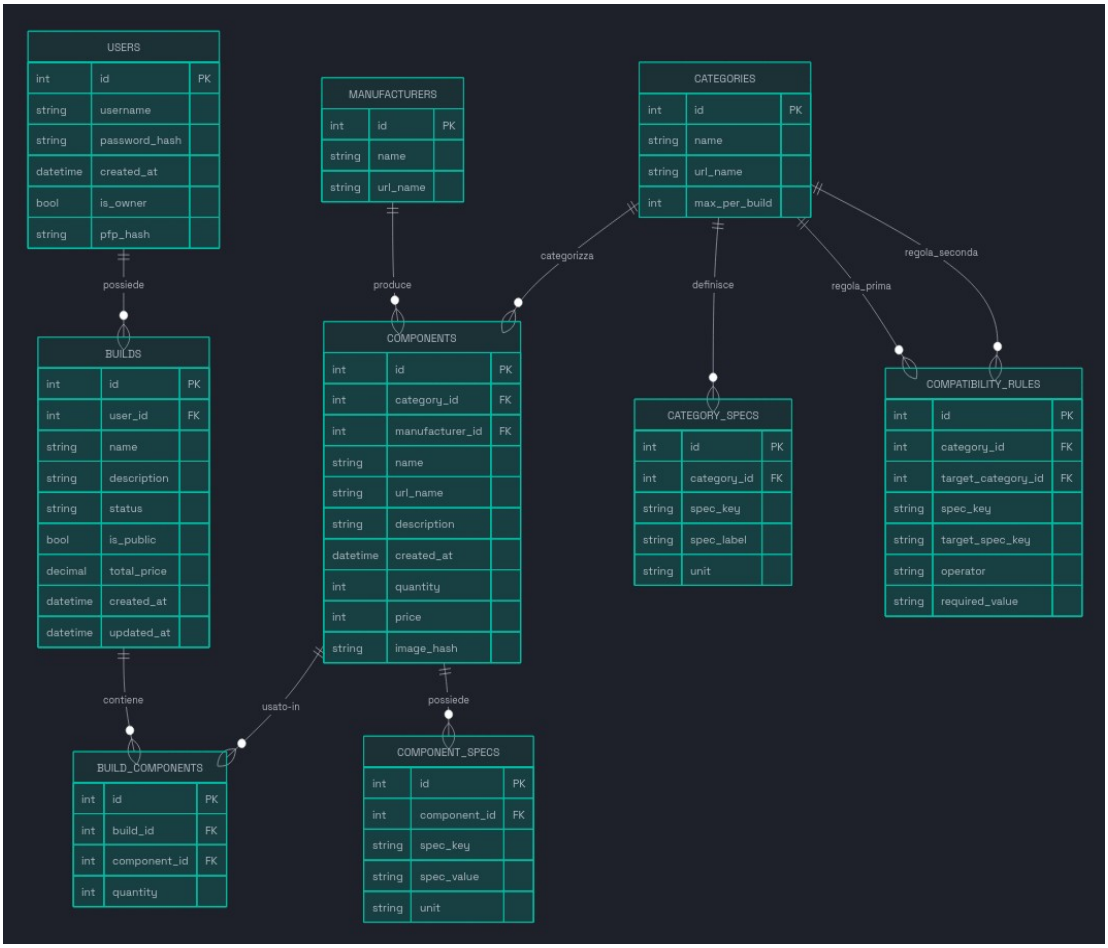
5. DATABASE

5.1 Schema logico

Di seguito lo schema logico completo delle tabelle con chiavi primarie (PK) e chiavi esterne (FK):

Tabella	Attributi
USERS	id PK, username, password_hash, created_at, is_owner, pfp_hash
BUILDS	id PK, user_id FK, name, description, status, is_public, total_price, created_at, updated_at
BUILD_COMPONENTS	id PK, build_id FK, component_id FK, quantity
COMPONENTS	id PK, category_id FK, manufacturer_id FK, name, url_name, description, created_at, quantity, price, image_hash
CATEGORIES	id PK, name, url_name, max_per_build
CATEGORY_SPECS	id PK, category_id FK, spec_key, spec_label, unit
COMPONENT_SPECS	id PK, component_id FK, spec_key, spec_value, unit
COMPATIBILITY_RULES	id PK, category_id FK, target_category_id FK, spec_key, target_spec_key, operator, required_value
MANUFACTURERS	id PK, name, url_name

5.1 Schema ER semplice



5.2 Schema ER completo